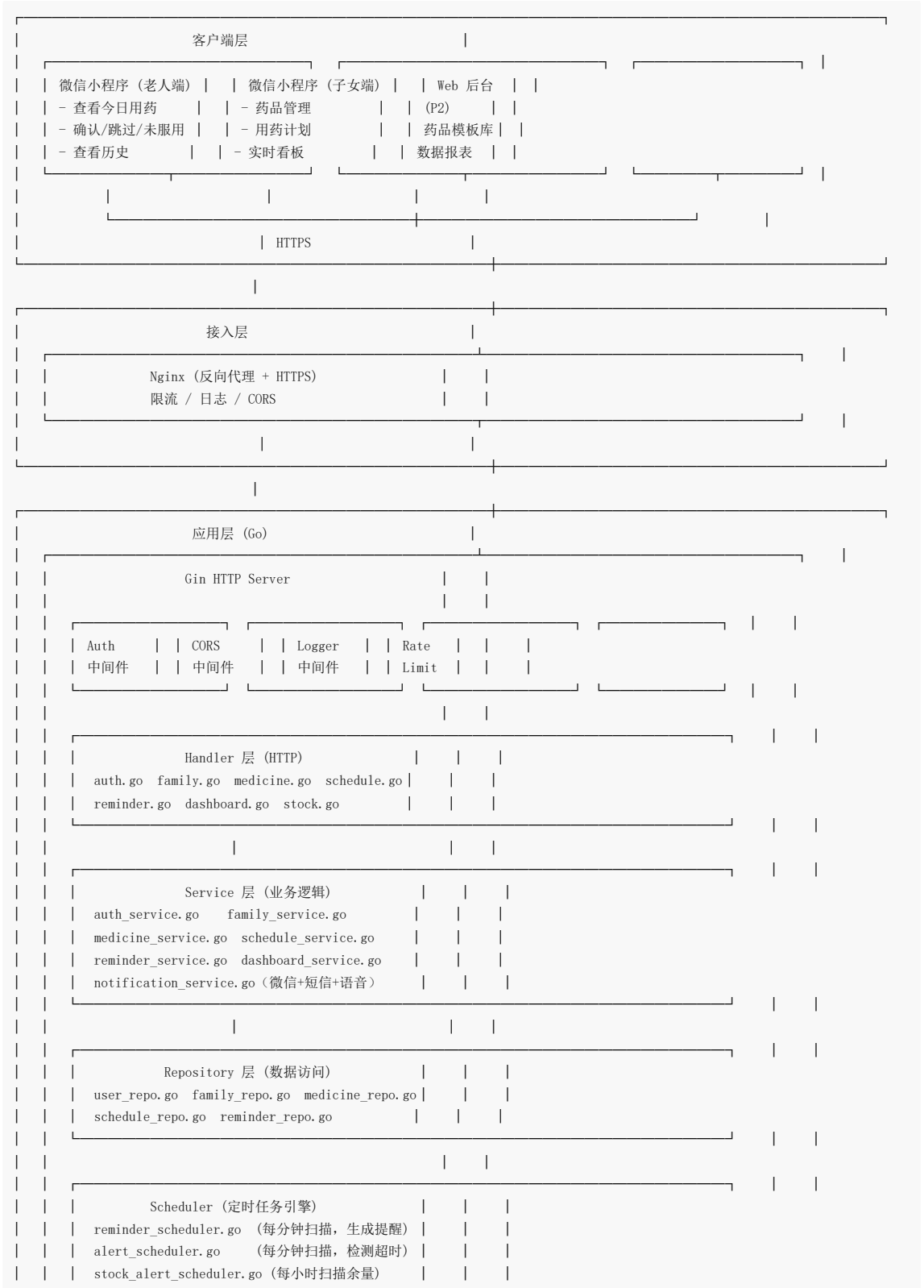
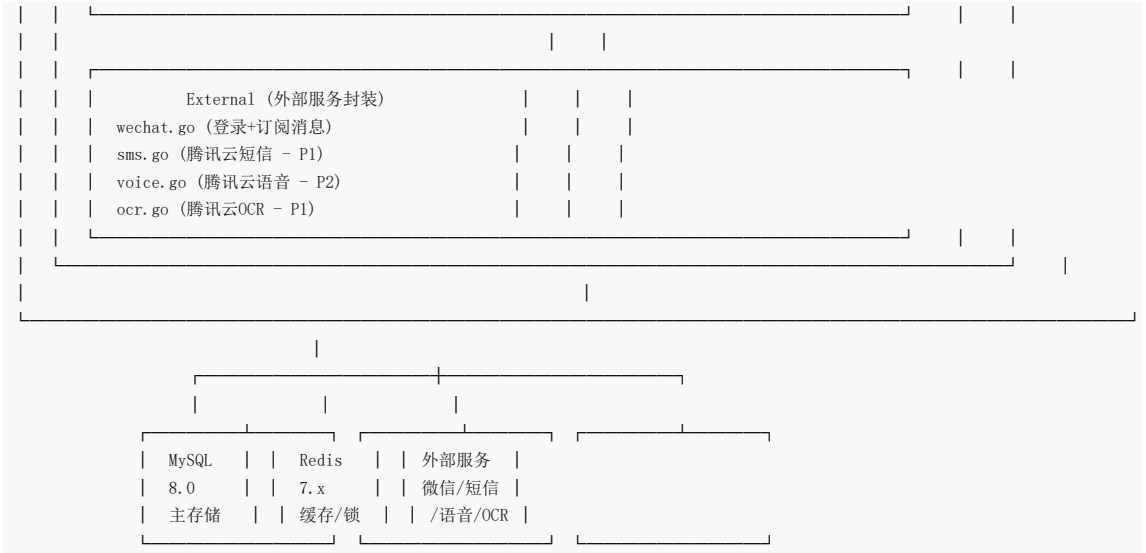
系统架构文档 —「按时吃药」

> *版本: v1.0 | 作者: 张架构 | 日期: 2026-07-06*

---

1. 整体架构图





---

2. 技术栈清单

层级	技术	版本	选型理由
语言	Go	1.22+	CEO 指定，性能优异，部署简单（单二进制）
Web 框架	Gin	v1.10	社区最活跃的 Go Web 框架，中间件生态完整
ORM	GORM	v2	Go 生态首选 ORM，AutoMigrate 加速 MVP 迭代
数据库	MySQL	8.0	微信小程序后端标配，腾讯云托管兼容好
缓存	Redis	7.x	JWT 黑名单 + 定时任务分布式锁
定时任务	robfig/cron	v3	Go 原生实现，秒级精度，单机够用
JWT	golang-jwt	v5	社区标准库
配置管理	viper	v1	多环境配置（dev/staging/prod）
日志	zap	v1	高性能结构化日志
微信 SDK	wechat-go 或自封装	-	登录 + 订阅消息
短信	腾讯云 SMS SDK	-	P1
语音	腾讯云语音通知 SDK	-	P2
OCR	腾讯云 OCR SDK	-	P1
部署	腾讯云轻量服务器	-	初期单机部署，后续可升配或容器化

---

3. 模块划分与职责

```
backend/internal/
├── handler/          ← HTTP 层：参数绑定、校验、响应封装
│   ├── auth.go       → POST /auth/login, POST /auth/refresh
│   ├── family.go     → 家庭组 CRUD
│   ├── medicine.go   → 药品 CRUD
│   ├── schedule.go   → 用药计划 CRUD
│   ├── reminder.go   → 确认/未服用/跳过/撤销
│   ├── dashboard.go  → 今日看板、概览
│   ├── stock.go      → 余量查询+校准
│   └── alert.go      → 告警记录查询
│
├── service/          ← 业务逻辑层（纯 Go，不依赖 HTTP）
│   ├── auth_service.go       → 微信 code → openid → JWT
```

```
│   ├── family_service.go     → 家庭组创建/邀请/角色
│   ├── medicine_service.go   → 药品管理+关联校验
│   ├── schedule_service.go   → 用药计划+明细管理
│   ├── reminder_service.go   → 确认/撤销/余量扣减
│   ├── dashboard_service.go  → 看板数据聚合
│   ├── notification_service.go → 消息分发（微信→短信→语音 三级）
│   └── alert_service.go      → 超时检测+告警升级
│
├── repository/       ← 数据访问层（GORM）
│   ├── user_repo.go
│   ├── family_repo.go
│   ├── medicine_repo.go
│   ├── schedule_repo.go
│   ├── reminder_repo.go
│   ├── stock_repo.go
│   └── audit_repo.go
│
├── model/            ← 数据模型（GORM Model）
│   ├── user.go
│   ├── family.go
│   ├── medicine.go
│   ├── schedule.go
│   ├── reminder.go
│   └── stock.go
│
├── middleware/        ← Gin 中间件
│   ├── auth.go       → JWT 验证 + 用户注入 Context
│   ├── family.go     → 家庭组成员鉴权（是否属于该 family）
│   ├── role.go       → 角色校验（非 elder 才能管理药品）
│   ├── logger.go     → 请求日志（zap）
│   ├── ratelimit.go  → API 限流（令牌桶）
│   └── recovery.go   → Panic 恢复
│
├── scheduler/        ← 定时任务
│   ├── reminder_scheduler.go   → 每分钟扫描用药计划，生成当日 reminders
│   └── alert_scheduler.go      → 每分钟扫描 reminders，检测超时并升级告警
│
├── external/         ← 外部服务接口（interface 定义 + 实现）
│   ├── wechat.go     → WechatService interface
│   ├── sms.go        → SMSService interface
│   ├── voice.go      → VoiceService interface
│   └── ocr.go        → OCRService interface
│
├── config/
│   └── config.go     → viper 配置加载
│
└── pkg/
    ├── errcode/      → 错误码枚举
    ├── response/     → 统一响应封装
    ├── jwt/          → JWT 生成/解析
    ├── crypto/       → AES 手机号加密
    └── utils/        → 通用工具函数
```

---

4. 部署架构

4.1 开发环境

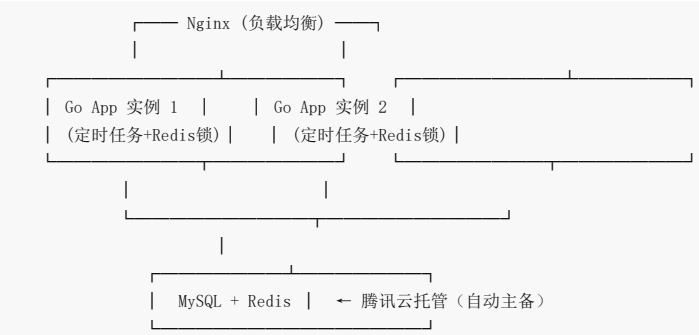
```
┌──────────────────┐    ┌──────────────────┐    ┌──────────────────┐
│  Go App  │──│  MySQL  │──│  Redis   │
│  :8080   │    │  :3306   │    │  :6379   │
└──────────────────┘    └──────────────────┘    └──────────────────┘
```

全部本地 Docker Compose 一键启动

4.2 生产环境（初期）

```
腾讯云轻量应用服务器 (2C4G)
├── Nginx (:80/:443 → :8080)
├── Go App (:8080)            ← 单实例
├── MySQL 8.0 (:3306)         ← 腾讯云数据库 MySQL（托管）
└── Redis 7.x (:6379)         ← 腾讯云数据库 Redis（托管）
```

4.3 生产环境（扩展后）



---

5. 关键技术难点

5.1 提醒调度的并发模型

挑战:

- 每个用户可能有 3-5 个用药时间点
- 1000 个用户 × 3 个时间点 = 3000 条提醒/天
- 需要在 ±1 分钟内准时生成并推送

方案:

```
reminder_scheduler（每分钟执行一次）:

1. 获取 Redis 分布式锁 (key: "scheduler:reminder", TTL 50s)
2. 查询 medicine_schedules WHERE time_of_day BETWEEN (NOW() - 1min) AND (NOW() + 1min) AND is_active = 1
3. 对每条 schedule:
   a. 检查是否已生成今日提醒 (reminders WHERE schedule_id + remind_date = today)
   b. 未生成 → 创建 reminder + reminder_items（快照药品信息）
   c. 已生成 → 跳过
4. 查询 reminders WHERE remind_time <= NOW() AND status = 'pending'
5. 对每条 reminder → 调用微信订阅消息推送 → status = 'sent'
6. 释放锁
```

性能计算:

- 1000 个用户，单次扫描 < 500ms
- 微信消息推送并发控制在 50 QPS（微信限制），使用 rate limiter

5.2 微信订阅消息频率限制

限制: 微信订阅消息对同一用户同一模板有频次限制，且用户每次授权只能用有限次数。

应对策略:

策略	说明
精准推送	仅在用药时间点推送，不滥用
合并推送	同一时间点的多种药品合并为一条消息
授权引导	子女端显示「老人订阅状态」，过期前 3 天引导重新授权
兜底通道	微信消息失败 → 短信（P1）→ 语音电话（P2）

静默检测	每晚 22:00 巡检当日授权状态，异常通知子女

5.3 数据脱敏方案

```
脱敏层级:

1. API 响应层（Middleware 自动处理）:
   - phone: "138****1234"（log 中自动替换）
   - 不做额外处理即返回，Service 层返回完整数据

2. 数据库存储层:
   - phone: 仅存后 4 位（phone 字段）
   - phone_enc: AES-256-GCM 密文存储完整手机号
   - openid: 不通过 API 返回，仅在内部使用

3. 日志层:
   - zap 自定义 Encoder，自动脱敏手机号/身份证字段
   - JSON 类型的请求体自动扫描并替换敏感 key

4. 审计日志:
   - audit_logs.detail 中敏感字段写入前替换为 "***"
```

5.4 提醒生成与跨天边界

边界场景:

- 用药计划 23:30 → 提醒生成时间在当天 23:30
- 凌晨 00:30 的提醒 → 属于新的一天

方案:

- reminders 按 `remind_date` 分区查询
- scheduler 每次执行时，以当前时间的日期为基准，扫描 ±1min 窗口
- 跨天时（23:59 → 00:01），自动生成新一天的提醒

---

6. 第三方服务选型

服务类型	候选 A	候选 B	推荐	理由
微信 SDK	自封装（官方 API）	silent-wx/wechat-go	**自封装**	依赖简单，只用到 login + 订阅消息两个接口，不值得引入第三方库
短信	腾讯云 SMS	阿里云短信	**腾讯云 SMS**	与微信小程序同属腾讯生态，到达率更高
语音电话	腾讯云语音通知	阿里云语音通知	**腾讯云语音通知**	同上，模板审核速度腾讯更快
OCR	腾讯云 OCR	百度 OCR	**腾讯云 OCR**	药品包装识别准确率腾讯优于百度
对象存储	腾讯云 COS	阿里云 OSS	**腾讯云 COS**	药品图片存储，同生态

---

7. 安全方案

7.1 认证流程

```
用户点击微信登录
  → 小程序调用 wx.login() 获取 code
  → POST /api/v1/auth/login { code }
  → 后端用 code 换取 openid + session_key（微信接口）
  → 查找或创建用户
  → 生成 JWT Token (sub=user_id, exp=7天)
  → 返回 access_token + refresh_token
```

7.2 鉴权机制

```
中间件 auth.go:
  1. 提取 Authorization header
  2. 解析 JWT → 提取 user_id
  3. 验证 Token 是否在 Redis 黑名单中
  4. Context.Set("user_id", user_id)
  5. 后续 Handler/Service 从 Context 获取当前用户

中间件 family.go:
  1. 从 URL 提取 family_id
  2. 查询 family_members WHERE family_id = ? AND user_id = ?
  3. 不存在 → 403 非家庭成员

中间件 role.go:
  1. 查询当前用户在家庭组中的角色
  2. elder 角色调用管理类接口（药品/计划增删改）→ 403
```

7.3 数据加密

数据	加密方式	密钥管理
手机号完整值	AES-256-GCM	环境变量 ENCRYPTION_KEY，定期轮换
JWT 签名	HMAC-SHA256	环境变量 JWT_SECRET
数据库连接	TLS	腾讯云 MySQL 自动启用

7.4 API 限流

接口类型	限制	窗口	说明
登录	10 次	1 分钟	防止恶意调用
通用接口	100 次	1 分钟	单用户
确认接口	30 次	1 分钟	防止重复提交
定时任务推送	50 次	1 秒	微信 API 限制

---

8. 关键风险点（供王测试重点测试）

编号	风险	等级	测试重点
R1	提醒重复推送（无分布式锁）	高	多实例下同一提醒是否只推送一次
R2	跨天提醒边界	高	23:59 和 00:01 的提醒正确性
R3	确认后余量未扣减	高	确认 → 查询余量 → 确认 → 再次查询
R4	JWT 过期处理	中	Token 过期后前端行为是否正确
R5	微信消息推送失败重试	中	模拟微信 API 返回错误 → 重试机制
R6	并发确认同一条提醒	中	两人同时点「已服用」→ 幂等处理
R7	药品删除后历史记录完整性	中	删除药品 → 历史提醒中的药品名快照是否完整
R8	短信发送频率控制	中	短时间内多条超时提醒 → 不重复发短信

> 📋 ***系统架构文档完成。总计 4 份技术文档已全部产出，等待 CEO 审批。***